MATLAB Simulation Exercise

Statistical Signal Processing II

RNN

Group 09

Student 1: BRIAN KIPROP KIBOR 2508789

Student 2: SUDIP SAHA 2508788

Student 3: SHAMIM AHMED 2509740

University of Oulu

Finland

February 06, 2026

Theoretical Background

In many wireless systems, the channel changes over time due to motion and multipath effects. When sampled at the symbol rate, the channel can be modeled as a finite impulse response (FIR) filter whose tap coefficients vary with time. The received signal observation model,

$$x[n] = v^T[n]h[n] + w[n] \tag{1}$$

where $v[n]$ is the transmitted signal vector, $h[n]$ the channel impulse response coefficient and $w[n]$ is noise. Our goal is to estimate the channel taps $h[n]$ at every time step. This is done using the state space model.

$$h[n] = Ah[n-1] + u[n], \tag{2}$$

where A is the state transition matrix and u$[n]$is zero-mean white Gaussian process noise with covariance matrix Q. This means the current channel is approximately the previous one plus small random change.

With the linear state equation (2) and linear observation equation (1) defined above, the channel estimation problem is modelled using Kalman filtering framework. The Kalman filter provides a recursive minimum mean-square error (MMSE) estimator for the channel coefficient vector h$[n]$ by combining prior state predictions with noisy observations. At each time step n, the Kalman filter operates in two main phases i.e. prediction and correction [1].

In the prediction step, the filter propagates the previous state estimate and its error covariance forward in time using the state-space model. As a result, the predicted channel estimate is provided by;

$$\hat{h}[n|n-1] = A\hat{h}[n-1|n-1] \tag{3}$$

while the prediction error covariance,

$$M[n|n-1] = AM[n-1|n-1]A^T + Q \tag{4}$$

The Kalman gain which is used in the correction step is calculated as;

$$K[n] = \frac{M[\,n \mid n-1\,]v[n]}{\sigma^2 + v^T[n]M[\,n \mid n-1\,]v[n]} \tag{5}$$

In the correction phase, the prediction is updated using the new observation $x[n]$, weighted by the Kalman gain;

$$\hat{h}[\,n \mid n\,] = \hat{h}[\,n \mid n-1\,] + K[n]\big(x[n] - v^T[n]\hat{h}[\,n \mid n-1\,]\big) \tag{6}$$

The updated error covariance is provided as:

$$M[\,n \mid n\,] = (I - K[n]v^T[n])M[\,n \mid n-1\,] \tag{7}$$

Using these equations, the Kalman filter recursively predicts the next channel from the previous estimate corrects it using the new received sample. The MMSE estimate tracks the fading channel efficiently. However, it requires a known state-space model i.e. matrices A, Q, and noise statistics to be known.

However, in practice, the exact channel dynamics may be unknown or too complex to model accurately. The matrices A, Q, and the noise statistics required by the Kalman filter may not be available or may change with the environment. In such cases, instead of relying on a predefined state–space model, the estimator can learn the channel behavior directly from data using a Recurrent Neural Network (RNN).

Similar to the Kalman filter, the RNN treats channel estimation as a sequential problem. At each time step n, the receiver observes the noisy sample $x[n]$ and knows the transmitted symbols $v[n]$, $v[n\text{-}1]$ ... These are combined into an input vector as below.

$$z[n] = [x[n], v[n], v[n-1], \dots] \tag{8}$$

Then vector $z[n]$ is fed into the RNN.

The RNN maintains an internal hidden state s [n] s[n] that acts as memory of past observations and evolves recursively according to standard RNN formulations [2].

$$s[n] = f_\theta(s[n-1], z[n]) \tag{9}$$

where $f_\theta(\cdot)$ is a nonlinear function with learnable weights θ. Using the updated hidden state, the RNN produces the channel estimate through an output mapping

$$\hat{h}[n] = g_\theta(s[n]) \tag{10}$$

Therefore, the hidden state implicitly captures the time variation of the channel, playing a role similar to predicted state in the Kalman filter, but without the need to assume a specific linear model. During training, the network weights are updated by minimizing the mean-square error between the true channel and the estimated channel. This leads to the loss function defined as;

$$L = \frac{1}{N}\sum_{n=1}^{N} \| h[n] - \hat{h}[n] \|^2 \tag{11}$$

Through training, the RNN gradually learns how the channel evolves over time by observing many examples. Instead of relying on a predefined mathematical model, it discovers the underlying temporal patterns directly from data.

Thus, an RNN can be viewed as a nonlinear recursive, data-driven estimator. It uses the information stored in its hidden state together with the newest observation to continually refine its channel estimate [2]. While the Kalman filter depends on an explicitly defined state-space model, the RNN learns this model implicitly. This makes RNN-based estimation

more flexible and better suited to situations where the channel statistics are unknown or change rapidly.

Problem Setup

In this lab, a Recurrent Neural Network (RNN) is used to estimate a two-tap time-varying FIR channel. The objective is to learn the channel coefficients directly from data and evaluate how well the network tracks channel variations over time.

The channel follows the same state–space model used in Lab 2. The transmitted signal $v(n)$ passes through a tapped-delay line, where each delayed branch is weighted by a time-varying coefficient $h_n[k]$. The weighted signals are summed to form the noiseless output, and additive white Gaussian noise $w(n)$ is added to produce the observed signal $x(n)$.

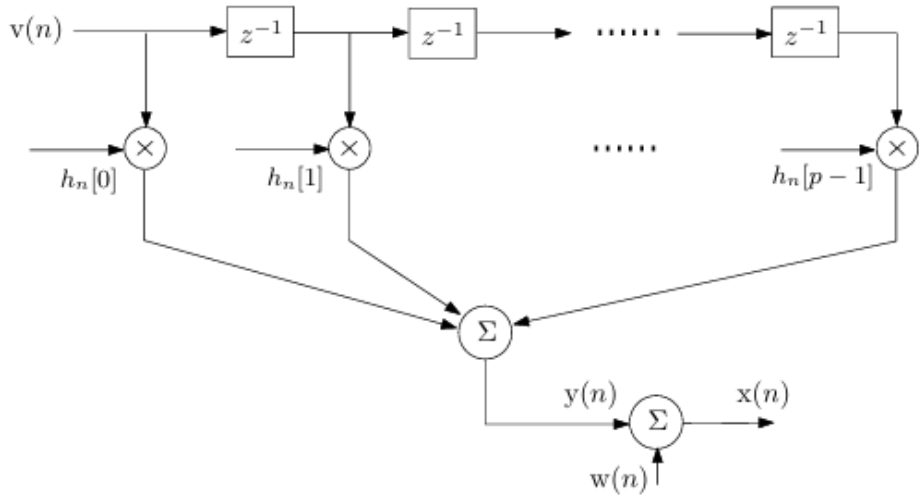

Fig. 1: FIR low-pass filter representation of the channel.

Results and Discussion

i. Channel Tracking and MSE N=100

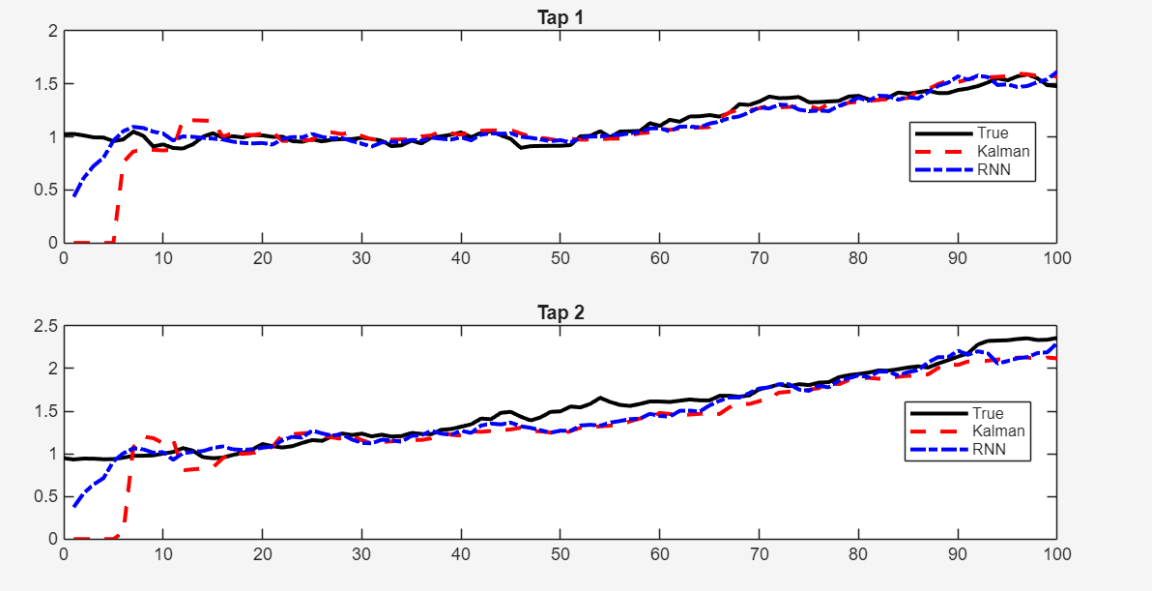

Fig. 2: Channel tracking comparison between Kalman and RNN **N=100**

During the initial periods i.e. $n \approx (1 - 10)$, the Kalman filter exhibits poor performance, as shown by its estimates being almost almost flat and far from the true channel values, while the RNN starts much closer to the true channel and improves rapidly. This is because the Kalman filter is initialized with a zero state and an identity covariance matrix, meaning it begins with no prior knowledge of the channel and requires several observations before it can converge. As a result, a large initial estimation error is observed. In contrast, the RNN has been trained on many similar channel realizations and learns the structure of the channel evolution. Even without an initialization, it is able to provide a good estimate initially.

In the steady-state region i.e. $n \approx (15 - 80)$, the estimates produced by the Kalman filter, the RNN, and the true channel nearly overlap, with minor deviations during periods of faster channel variation. At this stage, the Kalman filter has fully converged and operates at the MMSE bound, leveraging the exact knowledge of the system parameters i.e. matrices A *and* Q with noise variance σ^2. The RNN closely follows the Kalman filter's behavior, showing only slight lag during rapid changes and no signs of divergence.

In the final stages i.e. $n \approx (80 - 100)$, the differences between the two estimators become very small. The RNN estimates appear slightly smoother, whereas the Kalman filter responds more sharply to measurement noise. This difference reflects a bias–variance tradeoff between the two approaches: the Kalman filter achieves minimum variance estimates by optimally incorporating measurements according to the model, while the RNN introduces a

small bias due to its learned temporal correlations but gains robustness through implicit smoothing.

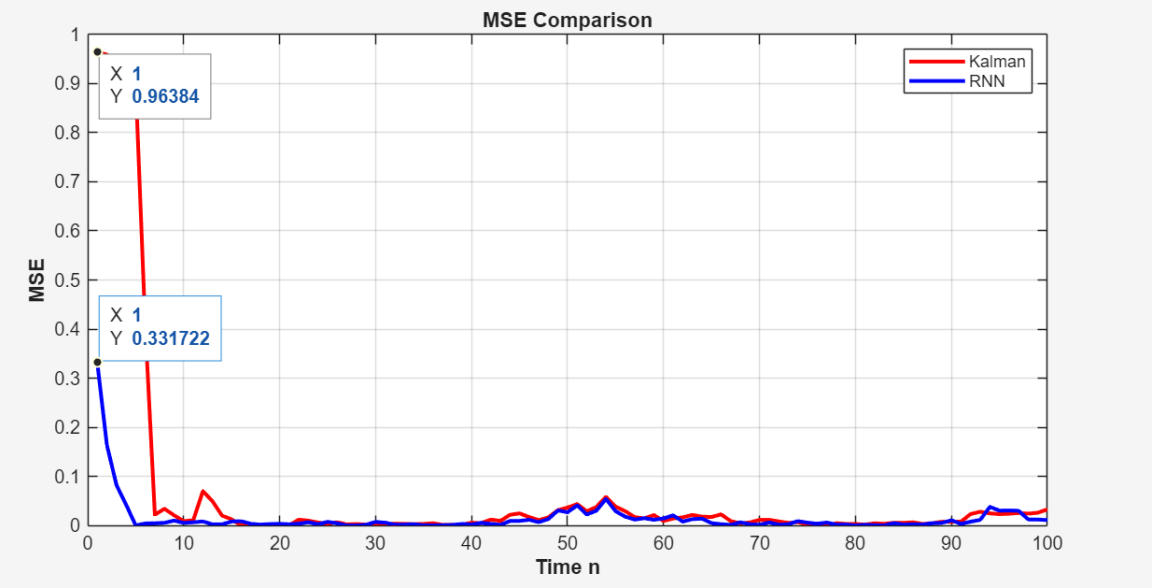

Fig. 3: MSE comparison between Kalman and RNN **N=100**

Figure 3. above shows the MSE comparison between Kalman and RNN. The plot highlights the different convergence between Kalman filter and the RNN over time. At the initial time instant ($n = 1$), the Kalman filter has a very large mean squared error (0.96), which is a direct consequence of the zero initial state and lack of prior information about the channel. In contrast, the RNN starts with a very low MSE (0.332), indicating that it is able to leverage the statistical knowledge gained during training to produce a more accurate initial estimate. As time goes by, the Kalman filter rapidly converges, and its MSE drops sharply within the first few samples, eventually reaching a low. The RNN also converges quickly and faster than Kalman, with its MSE decreasing smoothly. After the transient phase, both estimators achieve similar MSE levels, with only small fluctuations corresponding to faster channel variations and noise effects. These results confirm that while the Kalman filter is optimal in steady state, the RNN provides superior performance during the initial transient and is able to match the Kalman filter's accuracy after convergence, demonstrating effective learning of the channel dynamics from data.

ii. Channel Tracking and MSE N=300

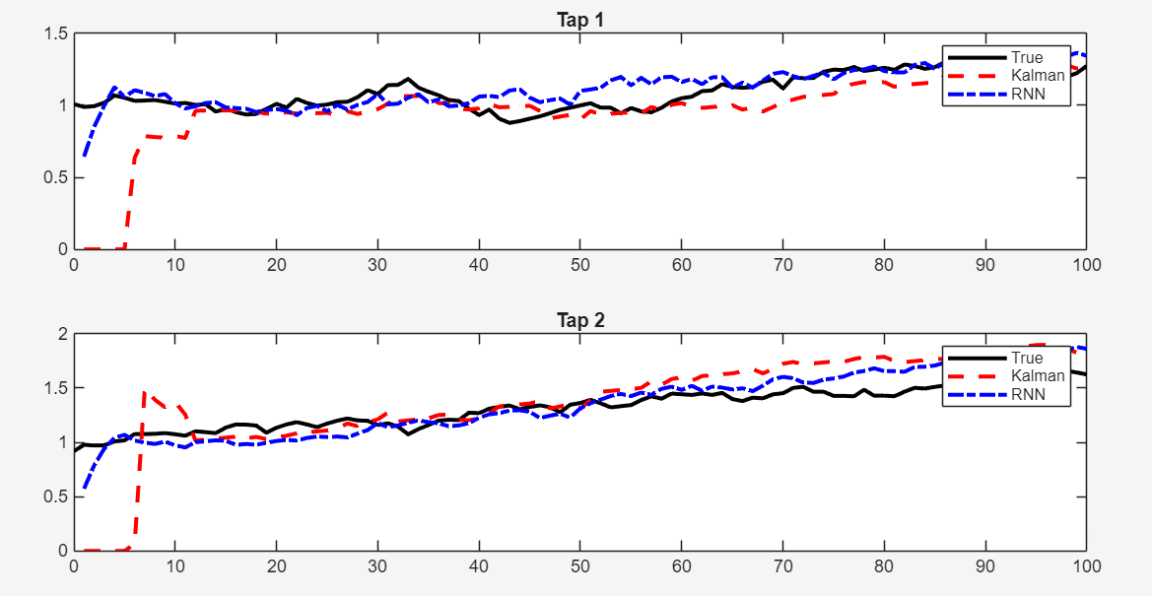

Fig. 4: Channel tracking comparison between Kalman and RNN **N=300**

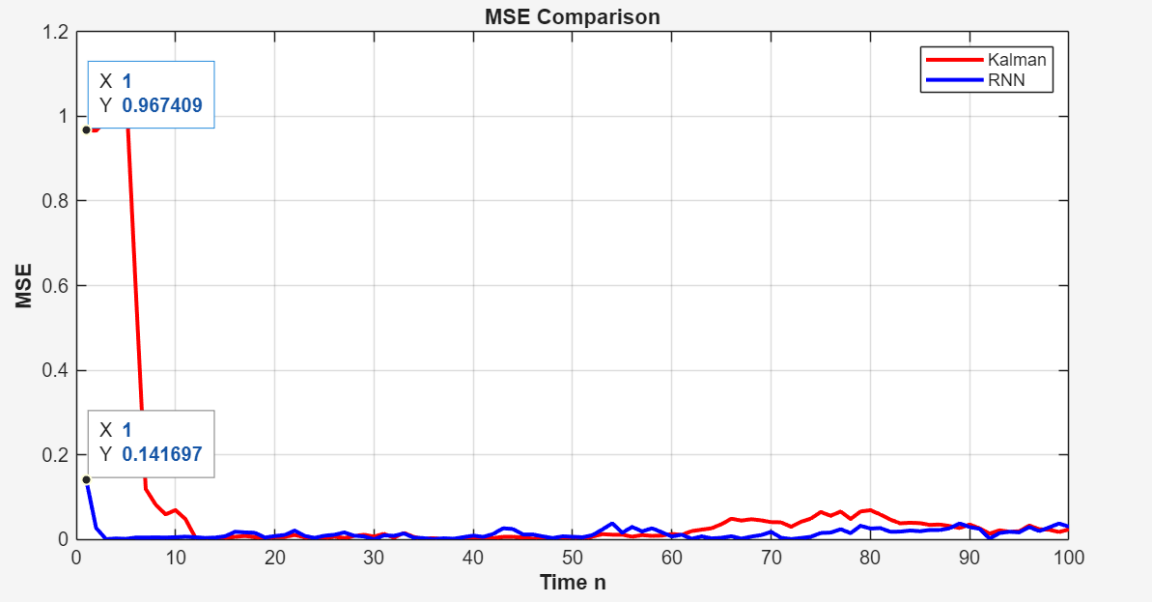

Fig. 5: MSE comparison between Kalman and RNN **N=300**

When we increased the number of training sequences to 300, the tap-tracking plots and the MSE curves remained very similar to the case with 100 training sequences. This indicates that our RNN has already learned the dominant channel dynamics with the smaller training set. Since the channel model is slowly varying, a moderate amount of training data is sufficient for the network to generalize well. As a result, increasing the training data further does not

lead to a large improvement in channel tracking or MSE reduction, because the RNN performance is already close to the optimal Kalman filter benchmark.

The main difference is observed in the MSE plot, the initial error of the RNN is slightly reduced and the MSE curve appears smoother, with fewer fluctuations. This suggests improved robustness and reduced variance. Overall, this result confirms that the RNN converges quickly to a near-optimal estimator with relatively few training sequences, and increasing the dataset size beyond this point provides only incremental gains.

iii. Channel Tracking and MSE N=1000

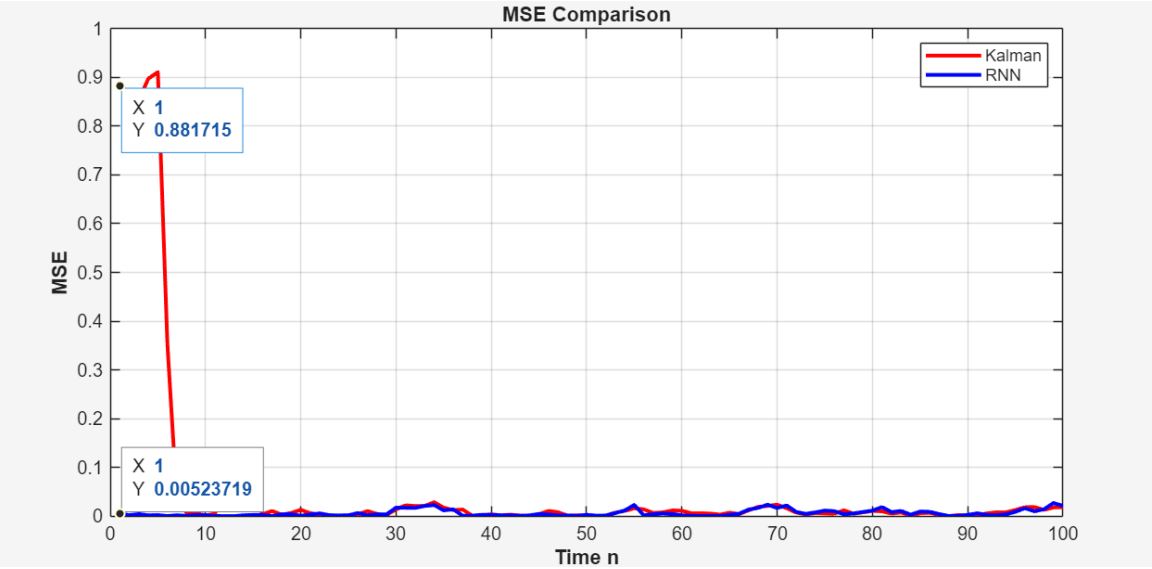

Fig. 6: MSE comparison between Kalman and RNN **N=1000**

When we increase the training sequences to 1000, the RNN showed a clear improvement mainly in the initial transient. The initial MSE of the RNN becomes almost zero, while the Kalman filter still exhibits a large error due to its zero initialization. This indicates that our RNN has learned a strong prior on the channel from data and can produce accurate estimates immediately. After convergence, both the RNN and the Kalman filter achieve similarly low steady-state MSE, with only small fluctuations which can be attributed to noise. Compared to 100 and 300 training sequences, the main gain from increasing the data size appears at early time instants, while steady-state performance remains largely unchanged. <<NB: Used 1000 to see the difference.>>

iv. Quantitative Performance Comparison

To complement the visual comparison done above, we now switch between the different Recursive Neural Network models to compare their estimation accuracy, convergence and learning dynamics.

Table 1. Performance Comparison (300 Samples, lr = 0.001, optimizer= Adam)

Criterion	**Kalman Filter**	**GRU**	**LSTM**
Initial MSE	0.9212	0.1565	0.2769
Convergence time (Samples)	12	3	4
Steady State MSE	0.0170	0.0273	0.0222
Average MSE	0.0626	0.0114	0.0243

From the table 1, the initial MSE shows us the difference between model based and data driven estimators. The Kalman filter exhibits a large initial error (0.9212) due to its zero initialization and lack of prior knowledge about the channel. In contrast, both Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) start with significantly lower initial MSE values, indicating that the networks leverage patterns learned during training to provide accurate estimates from the first-time step. Among the two, the GRU achieves the lowest initial MSE, suggesting a stronger learned prior. The is further cemented by convergence time. While the Kalman filter requires around 12 samples to converge, both GRU and LSTM converge within 3 and 4 samples respectively. This rapid convergence demonstrates the ability of the RNNs to learn the channel dynamics and immediately track the channel without a long transient phase.

In terms of steady-state MSE, the Kalman filter achieves the lowest value (0.0170), consistent with the fact it is MMSE-optimal when the state-space model and noise statistics are perfectly known. Which can be said to be known in the steady state. Both GRU and LSTM perform well but not as Kalman and thus termed as near optimal. The average MSE over the entire sequence is better among the RNN based estimators with GRU particularly achieving the lowest overall error. This is due to the fact that RNNs have improved transient performance compared to the Kalman filter.

Overall, the results indicate that the Kalman filter outperforms RNNs in steady state when the channel model is accurately known, whereas RNN-based estimators, especially GRU, offer superior initial performance and faster convergence.

Table 2. Performance Comparison (300 Samples, lr = 0.001, optimizer= RMSProp)

Criterion	**Kalman Filter**	**GRU**	**LSTM**
Initial MSE	0.9575	0.0568	0.1034
Convergence time (Samples)	10	2	2
Steady State MSE	0.0067	0.0260	0.0204
Average MSE	0.0529	0.0120	0.0128

From the table 2, similar trends to the Adaptive Moment Estimation (Adam) optimizer case are observed, with improved convergence behavior for our RNN-based estimators. The Kalman filter exhibits a large initial MSE (0.9575) due to its zero initialization, while both GRU and LSTM start with much lower initial errors, confirming that the networks exploit learned temporal patterns to provide accurate estimates from the first-time step. The GRU achieves the lowest initial MSE among the RNNs.

In terms of convergence, Root Mean Square Propagation (RMSProp) enables faster stabilization of the RNNs compared to Adam. While the Kalman filter converges in about 10 samples, both GRU and LSTM converge within 2 samples, indicating highly efficient learning of the channel dynamics. In steady state, the Kalman filter maintains the lowest MSE (0.0067), consistent with its MMSE-optimality when the channel model is known. Both RNNs achieve slightly higher but near-optimal steady-state MSE. The average MSE remains lower for the RNN-based estimators due to their superior transient performance. Overall, RMSProp provides stable training and fast convergence for both GRU and LSTM, reinforcing the advantage of data-driven estimators in rapidly time-varying channel conditions.

Table 3. Performance Comparison (300 Samples, lr = 5e-3, optimizer= SGDM)

Criterion	**Kalman Filter**	**GRU**	**LSTM**
Initial MSE	0.9575	0.0201	0.0034
Convergence time (Samples)	10	1	1
Steady State MSE	0.0067	0.0427	0.0235
Average MSE	0.0529	0.0188	0.0164

From table 3, using Stochastic Gradient Descent with Momentum (SGDM) with a higher learning rate (5e-3), both GRU and LSTM exhibit fast convergence, reaching steady behavior within a single sample. The also portray very low initial MSE values because momentum-based optimization strongly amplifies the learned prior of the networks, allowing them to produce accurate estimates immediately. However, this aggressive optimization also leads to degraded steady-state performance, particularly for the GRU, which shows a noticeably higher steady-state MSE. Compared to GRU, the LSTM maintains better stability and achieves lower steady-state and average MSE, indicating greater robustness to momentum-driven updates. Overall, while SGDM enables rapid convergence, it introduces a tradeoff between convergence speed and steady-state accuracy, with LSTM handling this tradeoff more effectively than GRU.

Conclusion

In this lab, Recurrent Neural Network (RNN) was used to estimate a time-varying two-tap FIR channel and its performance compared with Kalman filter. The RNN was trained using sequential received observations and known pilot symbols to learn the temporal evolution of the channel and produce recursive channel estimates. Our results show that RNN-based estimators achieve significantly lower initial estimation error and much faster convergence than the Kalman filter. This behavior indicates that the RNN successfully learns the temporal structure of the channel dynamics from training data and is able to exploit this learned prior to produce accurate estimates from the first few samples. As the number of training sequences increases, the initial transient error of the RNN further decreases, while steady-state performance remains close to that of the Kalman filter.

In steady state, the Kalman filter consistently achieves the lowest mean-squared error, reflecting its MMSE-optimality when the channel model and noise statistics are accurately known. The RNN, while slightly inferior in steady state, remains near-optimal and exhibits smoother estimates due to its temporal averaging. Additionally, we did further experiments with different recurrent architectures and optimizers which further highlighted the importance of training stability, with gated architectures such as LSTM and GRU demonstrating robust performance across a range of settings.

Overall, the findings confirm that the Kalman filter is preferable when an accurate state-space model is available and steady-state accuracy is the primary objective. In contrast, RNN-based estimators offer clear advantages in terms of rapid convergence and robustness to unknown or changing channel dynamics, making them a great alternative for practical wireless environments where precise channel models may not be available.

References

[1] S. M. Kay, Fundamentals of Statistical Signal Processing: Estimation Theory. Prentice Hall International, 1993.

[2] K. P. Murphy, Machine Learning: A Probabilistic Perspective. Cambridge, MA, USA: MIT Press, 2012, Sec. 15.2.